

System Design Interview (Alex Yu) – Problem Summaries

A study reference covering the 12 design problems from *System Design Interview: An Insider's Guide, Second Edition*. Each entry is broken into the problem itself, the high-level architecture, the deep-dive details an interviewer would probe, and the core algorithms/data structures involved.

1. Rate Limiter

High-Level Problem

Build a server-side component that throttles how many requests a client can send to an API within a time window – to block denial-of-service abuse, control costs on metered third-party calls, and keep servers from being overloaded. It needs to be accurate, low-latency, memory-efficient, work across a fleet of distributed servers, communicate clearly to throttled clients, and fail without taking down the rest of the system.

High-Level Design

The limiter sits as middleware in front of API servers (or inside an API gateway), not on the client — client-side enforcement is trivially bypassed. Requests pass through the limiter first; if under budget they're forwarded on, otherwise the caller gets an HTTP 429. Usage counters live in a fast in-memory store like Redis rather than a database, using atomic increment-with-expiry operations.

Design Deep Dive

Throttling rules (per user, per IP, per endpoint) are defined in config files loaded into a cache. Throttled responses include headers telling the client its remaining quota and retry-after time. In a distributed deployment, two issues appear: race conditions when multiple servers concurrently read-then-write the same counter (solved with atomic scripting or Redis's sorted-set operations rather than locks), and the need to share limiter state across many limiter instances via a centralized store instead of sticky sessions. Further hardening includes deploying limiters at edge locations near users and monitoring both algorithm accuracy and rule strictness so they can be tuned.

Algorithms & Data Structures

- **Token bucket** — capacity-limited bucket refilled at a fixed rate; absorbs short bursts well.

- **Leaking bucket** — a FIFO queue drained at a constant rate; smooths traffic to a fixed output rate.
 - **Fixed window counter** — a simple per-interval counter; vulnerable to bursts at window edges.
 - **Sliding window log** — stores every request timestamp (e.g., in a Redis sorted set) for exact accuracy at higher memory cost.
 - **Sliding window counter** — a cheap hybrid blending current and previous window counts, weighted by position in the window.
-

2. Consistent Hashing

High-Level Problem

With a plain hash-and-modulo scheme for distributing keys or requests across servers, adding or removing even one server reshuffles nearly every key's assignment, causing a storm of cache misses or data movement. The goal is a scheme where scaling the server pool up or down only requires remapping a small, proportional slice of keys.

High-Level Design

Both servers and keys are hashed onto positions on a fixed circular hash space (a "ring"). To find which server owns a key, walk clockwise from the key's position until the first server is reached. Adding a

server only reclaims the keys between it and its counter-clockwise neighbor; removing one only reassigns the keys it owned to its clockwise neighbor — everything else on the ring is unaffected.

Design Deep Dive

The naive ring has two flaws: partition sizes between servers can be very uneven, and a small physical server count can produce a lopsided key distribution. Both are fixed with **virtual nodes** — each physical server is represented by many scattered points on the ring instead of one, so as virtual-node count grows, load variance across servers shrinks (empirically down to single-digit percentages with a couple hundred virtual nodes per server), at the cost of more memory to track them. When a server joins or leaves, the affected range is found by walking the ring backward from that server's position to the next server.

Algorithms & Data Structures

The hash ring is the core structure — conceptually a sorted circular list (in practice a sorted array or tree of ring positions) supporting an efficient "find the next server clockwise" lookup. Virtual nodes multiply the ring entries per server to control load balance. This technique underlies partitioning in Dynamo, Cassandra, and load balancers like Google's Maglev.

3. Key-Value Store

High-Level Problem

Design a distributed key-value database (`put(key, value)` / `get(key)`) holding small values (under 10 KB) at massive scale, staying available through node and network failures, scaling its server pool automatically, and offering tunable consistency — a simplified Dynamo/Cassandra/Bigtable-style system.

High-Level Design

A single in-memory hash table suffices for one server, but a real system spreads data across many nodes, which immediately raises the **CAP theorem** trade-off: because network partitions are unavoidable, the system must favor either consistency (CP) or availability (AP) when a partition occurs. Data is spread using consistent hashing (cheap to scale in/out), and each key is asynchronously replicated to N nodes found walking clockwise from its ring position, ideally spread across data centers for fault isolation.

Design Deep Dive

Consistency across replicas is tuned with quorum parameters — N (replica count), W (write acks required), R (read responses required); $W + R > N$ guarantees strong consistency at the cost of latency. Concurrent writes to different replicas create conflicting versions tracked with **vector clocks** (a per-

server version-counter list attached to each value), letting the system or client tell whether one version descends from another or genuinely conflicts. Failure detection uses a decentralized **gossip protocol** rather than one node unilaterally declaring another down. Temporary failures are handled with sloppy quorum (fall through to the next healthy nodes) and hinted handoff (a substitute node holds data temporarily, then hands it back once the original recovers); permanent failures are resolved by comparing **Merkle trees** between replicas to sync only out-of-date branches instead of re-transferring everything. On a single node, writes hit a commit log then an in-memory memtable, periodically flushed to an on-disk SSTable; reads check memory first, then use a **bloom filter** to cheaply rule out SSTables before touching disk.

Algorithms & Data Structures

Consistent hashing ring (partitioning), vector clocks (multi-version conflict tracking), Merkle trees (efficient replica-diffing), gossip protocol (failure detection), quorum consensus (tunable consistency), bloom filters (probabilistic pre-checks before disk reads), and SSTables (sorted, immutable on-disk files, the standard LSM-tree storage format).

4. Unique ID Generator (Distributed Systems)

High-Level Problem

Generate unique, roughly time-ordered, numeric 64-bit IDs across many machines at 10,000+ IDs/second, without relying on a single database's auto-increment, which doesn't scale across independent shards.

High-Level Design

Four candidates are weighed: multi-master DB replication with offset auto-increment (simple, but doesn't scale well across data centers and isn't time-ordered across servers); UUIDs (128-bit, fully decentralized, collision-free in practice, but too long, non-numeric-looking, and not time-sortable); a centralized ticket server (numeric and simple, but a single point of failure); and **Twitter's Snowflake approach** — the one adopted — which assembles an ID from meaningful bit-fields rather than hashing anything.

Design Deep Dive

The 64 bits break down as: a reserved sign bit; a 41-bit millisecond timestamp relative to a custom epoch (about 69 years of headroom); a 5-bit datacenter ID and 5-bit machine ID (32×32 combinations); and a 12-bit sequence number that resets every millisecond, capping a single machine at 4,096 IDs/ms. Because

the timestamp occupies the high-order bits, IDs sort naturally by creation time. Datacenter/machine IDs are fixed at startup and must be managed carefully to avoid collisions; clock synchronization across machines (e.g., NTP) becomes an implicit dependency.

Algorithms & Data Structures

No traditional data structure dominates here — the technique is **bit-field packing** (dividing a 64-bit integer into fixed-width timestamp/location/sequence sections) plus a simple counter for the per-millisecond sequence.

5. URL Shortener

High-Level Problem

Build a TinyURL-style service that converts long URLs into short aliases and redirects visitors back to the original — at ~100M new URLs/day (roughly 36.5 billion over 10 years, ~365 TB storage), with a 10:1 read/write ratio.

High-Level Design

Two REST endpoints: one to shorten a URL (POST), one to redirect from a short code (GET, returning an HTTP redirect). A **301** (permanent) redirect lets browsers cache the mapping and reduces load on the shortener; a **302** (temporary) redirect routes every

click through the service, useful when click analytics matter.

Design Deep Dive

Short codes are 7 characters from a 62-character alphabet (0–9, a–z, A–Z), since 62^7 comfortably exceeds the ~365 billion URLs the system must support. Two encoding strategies are compared: hashing the long URL and truncating it (requires collision handling — append a string and retry, cheaply pre-checked with a bloom filter) versus base-62 encoding a globally unique integer ID from an ID generator, which sidesteps collisions entirely and is the chosen approach. The shortening flow checks for an existing mapping, mints a new unique ID if needed, base-62 encodes it, and persists the pair; the redirect flow checks a cache before falling back to the database. A rate limiter guards the shortening endpoint against abuse.

Algorithms & Data Structures

Base-62 conversion (representing an integer ID in a 62-symbol alphabet), cryptographic/checksum hashing plus collision resolution as an alternative path, and bloom filters for cheap pre-checks before hitting the database.

6. Web Crawler

High-Level Problem

Build a scalable, "polite," robust, extensible crawler that discovers and downloads on the order of a billion HTML pages a month for search-engine indexing, retaining content for up to five years while skipping duplicates.

High-Level Design

Starting from seed URLs, the crawler loops between downloading pages and extracting new links, coordinated by pipeline stages: a **URL Frontier** holding URLs still to fetch; an HTML Downloader (backed by a DNS resolver); a Content Parser/validator; a "Content Seen?" check (comparing content hashes) to skip duplicates; a Link/URL Extractor; a URL Filter dropping disallowed or malformed links; and a "URL Seen?" check to avoid re-queuing visited URLs. Content sits mostly on disk with hot items cached in memory; visited URLs get their own storage.

Design Deep Dive

Breadth-first traversal via a FIFO frontier is preferred over depth-first, but plain BFS is both "impolite" (most links on a page point back to the same host, so parallel crawling can hammer one server) and indifferent to page importance. The frontier is redesigned with two layers: **front queues** bucketing URLs by priority (from signals like PageRank or update frequency), and **back queues**, one per host, enforcing a politeness delay via a mapping table and

queue router/selector. The frontier lives mostly on disk with an in-memory buffer, since it can hold hundreds of millions of URLs. The downloader respects `robots.txt` (cached to avoid repeat fetches), uses a cached DNS resolver so lookups don't stall the crawler, runs geographically close to crawled sites, and applies short timeouts against unresponsive servers. Robustness comes from consistent hashing to spread load across downloaders, persisted crawl state for restartability, and careful exception/data validation – plus defenses against "spider traps" (infinite link loops, mitigated with URL length limits) and low-value content like ads.

Algorithms & Data Structures

BFS graph traversal (the web modeled as a directed graph of pages/links) via FIFO queues; a priority scheme (front queues) layered over per-host politeness queues (back queues); bloom filters/hash tables for "URL Seen?" and duplicate-content checks; and consistent hashing to distribute crawl work.

7. Notification System

High-Level Problem

Deliver tens of millions of notifications a day (push, SMS, email) reliably and without excessive duplication, triggered by internal services or

scheduled jobs, respecting user opt-outs, in a "soft real-time" system where occasional delay under load is acceptable but data loss is not.

High-Level Design

Each channel has its own delivery mechanism — iOS push via Apple's APNS, Android via Firebase Cloud Messaging, SMS/email via third-party providers like Twilio/SendGrid — fronted by internal notification servers exposing APIs to upstream services. Those servers fetch the recipient/template data they need from a cache/database and hand delivery off to per-channel message queues, so a third-party outage on one channel doesn't block the others; worker pools pull from those queues and call the relevant provider API.

Design Deep Dive

Reliability rests on persisting every notification event to a log/database before attempting delivery (enabling retries) and a dedupe check on event ID to reduce, though not fully eliminate, duplicate sends — exactly-once delivery isn't achievable in a distributed system. The design layers in reusable **templates** (consistent formatting, less per-notification build time), a settings table for per-channel opt-in/out, rate limiting so one user isn't flooded, a retry mechanism (failed sends return to the queue, with alerting on persistent failure), authentication (appKey/appSecret) so only verified clients can trigger sends, and

monitoring of both queue depth (catching under-provisioned workers) and engagement metrics like open/click rates.

Algorithms & Data Structures

Nothing algorithmically exotic — the core techniques are message-queue-based decoupling (one queue per channel to isolate failures), an idempotency check keyed on event ID, and simple relational tables for templates and settings.

8. News Feed System

High-Level Problem

Let users publish posts and see a reverse-chronological aggregation of friends' posts (Facebook/Twitter/Instagram-style), supporting up to 5,000 friends per user and 10M daily active users, including media.

High-Level Design

Split into two flows: **feed publishing**, where a post is persisted and "fanned out" to friends' feeds, and **feed retrieval**, where a user's feed is assembled and returned. On publish, a post service saves the post while a fanout service determines the audience and pushes post/user ID references into a per-user feed cache via a message queue; a notification service

alerts friends. On read, the feed service simply reads the already-computed list of post IDs from cache.

Design Deep Dive

The key decision is the fanout model. **Fanout-on-write** (push) pre-computes each friend's feed at publish time — fast reads, but wasted work on inactive users and a "hotkey" problem for accounts with huge follower counts. **Fanout-on-read** (pull) computes the feed at read time — no wasted work or hotkeys, but slower reads. The chosen design is a hybrid: push for typical users, pull for celebrities/high-follower accounts, with consistent hashing helping spread the load such accounts generate. The fanout service pulls the friend graph from a graph database, filters by mute/privacy settings, and stores only IDs (not full objects) in the feed cache to control memory. Retrieval "hydrates" those IDs by joining against user and post caches, with media served from a CDN. The cache tier is layered: feed IDs, content, social graph, user actions (likes/replies), and counters.

Algorithms & Data Structures

No single dominant data structure — the key pattern is **push vs. pull fanout** (a precomputation trade-off) combined with consistent hashing for high-fanout accounts, and a graph database to model the friend/follow relationships fanout depends on.

9. Chat System

High-Level Problem

Support 1-on-1 and small-group (up to 100 members) text chat for 50M daily active users, with low delivery latency, an online-presence indicator, multi-device sync, and permanent chat history — a Messenger-style app.

High-Level Design

Clients never talk directly to each other; everything routes through a chat service. Client-to-server messages work fine over plain HTTP, but server-to-client "push" needs another mechanism — the design compares polling (simple, wasteful), long polling (holds the connection open until a message arrives, but has stickiness and disconnect-detection issues), and **WebSocket** (persistent, bidirectional), which is chosen for both directions. Services split into stateless ones (login, profile, etc.), the one stateful service (chat servers holding live WebSocket connections), and third-party push integration for offline delivery. A key-value store (chosen for low-latency random access and easy horizontal scaling, matching the pattern used by Messenger and Discord) holds chat history; generic account data stays relational.

Design Deep Dive

A service-discovery layer (e.g., Apache ZooKeeper)

assigns each connecting client to a chat server based on load/location. For 1-on-1 messages, the sending server obtains a message ID, writes to a sync queue and the KV store, then forwards to the recipient's connected chat server (or triggers a push notification if offline). Multi-device sync works by having each device track its own "current max message ID" and pull anything newer from the store. Group chat fans a message out into a per-recipient sync queue (an inbox model) — efficient for small groups but not scalable to large ones. Message IDs must be unique and time-sortable, via either a global Snowflake-style generator or a simpler generator unique only within a conversation. Online presence uses a heartbeat mechanism (rather than flipping status on every disconnect, which would flicker on flaky connections) and propagates to friends via a publish/subscribe channel per friend pair.

Algorithms & Data Structures

WebSocket-based persistent connections for push delivery; a distributed ID scheme (global Snowflake-style or per-conversation local counters) for message ordering; a key-value store for chat history; publish/subscribe channels for presence fanout; and a heartbeat/timeout mechanism for liveness detection.

10. Search Autocomplete System

High-Level Problem

Return the top 5 most popular query completions as a user types, in under ~100ms, for 10M daily active users generating tens of thousands of queries per second — a "top-k" ranking problem under a tight latency budget.

High-Level Design

Split into a **data-gathering service** that aggregates raw search logs into query-frequency data, and a **query service** that, given a prefix, returns the top-k matches. A naive version keeps frequencies in a relational table and issues a "top 5 where query starts with X" query — workable at small scale, but too slow as the dataset grows.

Design Deep Dive

The core optimization replaces the relational lookup with a **trie** (prefix tree) where each node also caches its own top-k most frequent completions, turning a lookup into "walk to the prefix node, return its cached list" — effectively $O(1)$ after bounding prefix length, instead of traversing and sorting an entire subtree. Since rebuilding on every keystroke is unaffordable, the trie is built offline: raw logs are aggregated (e.g., weekly) by worker processes into a frequency table, which is used to construct a new trie that replaces the old one in a trie cache (also persisted, either as a

serialized document snapshot or a hash-table representation where each prefix maps to a key). The query path reads from that cache, refilling it from the DB on a miss, and is further sped up with AJAX requests and browser-side caching. Node-level updates (instead of a full rebuild) require propagating new counts up through every ancestor, since ancestors cache their descendants' top queries. A filter layer sits in front of the cache to strip out unwanted suggestions before they persist. At very large scale, the trie is sharded — ideally not just alphabetically (uneven, since more words start with common letters) but by historical query-volume distribution per letter range.

Algorithms & Data Structures

The **trie (prefix tree)** is the centerpiece, augmented at each node with a cached, pre-sorted top-k list; supporting techniques include offline batch aggregation of logs into frequency counts, and a hash-table encoding of a trie (prefix → node data) as one durable-storage option.

11. YouTube

High-Level Problem

Design a large-scale video platform supporting fast uploads, smooth adaptive-quality streaming,

encryption, and global reach for millions of daily active users, while keeping an eye on CDN bandwidth costs, which can balloon quickly at scale.

High-Level Design

The system leans on existing cloud building blocks — blob storage for raw and transcoded video, and a CDN for playback — rather than reinventing them. Upload has two parallel paths: video bytes go to blob storage and are picked up by transcoding servers, while metadata (title, size, format, etc.) is written directly to a metadata DB/cache. Once transcoding finishes, encoded files move to CDN-backed storage and a completion event flows through a queue to workers that update metadata so the client learns the video is ready. Streaming pulls directly from the CDN using a standard protocol (e.g., MPEG-DASH or Apple HLS) so playback can start before the whole file downloads and adapt to viewer bandwidth.

Design Deep Dive

Transcoding — converting raw video into multiple resolutions/bitrates/codecs — is modeled as a **directed acyclic graph (DAG)** of tasks (splitting video/audio, encoding at multiple resolutions, generating thumbnails, adding watermarks), so independent tasks run in parallel and pipelines can be customized per creator. A preprocessor splits incoming video into small, independently playable chunks (Group-of-Pictures alignment) and builds the

DAG from configuration; a DAG scheduler breaks it into ordered task stages; a resource manager (task queue, worker queue, running queue, plus a scheduler matching tasks to workers) allocates compute; task workers execute the steps; results land in temporary storage until the pipeline completes. Layered optimizations include chunked/parallel resumable uploads, geographically distributed upload endpoints, decoupling pipeline stages with message queues for parallelism, pre-signed upload URLs for authorized-only uploads, DRM/encryption/watermarking to protect content, and — since viewership follows a long-tail distribution — serving only popular videos from the expensive CDN tier while less-popular content sits in cheaper bulk storage or is transcoded on demand. Error handling distinguishes recoverable failures (retry with backoff) from non-recoverable ones (abort and surface an error), with specific playbooks per component.

Algorithms & Data Structures

A **DAG** models and schedules the multi-stage transcoding pipeline, with priority queues driving the resource manager's task/worker matching; **GOP-based chunking** (splitting a video stream into independently decodable segments) underlies both parallel upload and adaptive streaming; content popularity (long-tail analysis) drives storage-tier placement.

12. Google Drive

High-Level Problem

Design a cloud file storage and sync service (upload/download, cross-device sync, version history, sharing, change notifications) for 10M daily active users with strong consistency requirements — a file must never appear different to two clients at once — plus a file-size cap and bandwidth constraints.

High-Level Design

Framed as an evolution from a single server (web server + DB + local disk) that quickly runs out of space, toward a decoupled architecture: a load balancer in front of stateless API servers; file bytes in cloud object storage (replicated across regions); metadata (users, files, versions, blocks) in a sharded/replicated relational database with a metadata cache; **block servers** doing the heavy lifting of splitting, compressing, and encrypting file content before storage; a notification service telling online clients when a file changes; and an offline backup queue holding changes for disconnected clients until they reconnect.

Design Deep Dive

To minimize bandwidth, files are split into fixed-size blocks (a few MB each), and only modified blocks are re-uploaded on an edit (**delta sync**) after compression

— re-uploading a whole file on every small change would be wasteful. Because the system requires strong consistency, the design favors a relational metadata store (natively ACID-compliant) over a NoSQL store that would need consistency logic bolted on, and cache entries are invalidated on every database write rather than left to converge eventually. The upload flow runs metadata registration and block upload in parallel, flipping the file's status from pending to uploaded once a completion callback confirms blob storage received it. Download/sync is triggered either by a live notification (client online) or a queued backlog the client drains on reconnect, reconstructing the file from only the blocks it doesn't already have. Long polling (rather than WebSocket) handles notifications since traffic is one-directional and infrequent. Storage costs are controlled via block-level de-duplication (identical blocks, identified by hash, stored once), capping or intelligently pruning old versions, and moving rarely accessed data to cheaper cold storage. Sync conflicts — two edits racing — are resolved first-write-wins, with the losing client shown both versions to merge or override manually.

Algorithms & Data Structures

Content-addressed block hashing (splitting files into blocks and identifying duplicates by hash) underlies both delta sync and storage de-duplication; a **delta-sync/diffing algorithm** (in the spirit of rsync)

determines which blocks actually changed; long polling handles notification delivery.

Reference: System Design Interview: An Insider's Guide, Second Edition, by Alex Yu (Chapters 4–15).